Национальный исследовательский университет
«Высшая школа экономики»
Факультет компьютерных наук
Департамент программной инженерии

Многопроцессорная симуляция шлюза

Отчёт по индивидуальному заданию 3
Архитектура компьютера и операционные системы
Вариант 47

Выполнил: Турицын Матвей Максимович
Группа: БПИ-252

Москва 2026

1 Постановка задачи

Разработаны три реализации имитации шлюза космической станции с взаимодействующими процессами. Все реализации используют C, Linux, разделяемую память POSIX и семафоры POSIX. Программа читает сценарий из файла, показывает события в консоли, сохраняет журнал и поддерживает завершение по сигналам.

Условие варианта 47

Шлюз имеет внутреннюю и внешнюю двери, ограниченную вместимость и систему изменения давления. Участники подают заявки на выход в космос и возвращение на станцию. Внутренняя дверь открывается только при нормальном давлении, а внешняя при низком. Две двери не могут быть открыты одновременно. Предусмотрены приоритет аварийного возвращения, длительности операций, расход ресурсов и отказы оборудования.

Выход в космос: подготовить нормальное давление, открыть внутреннюю дверь, впустить группу, закрыть дверь, снизить давление, открыть внешнюю дверь, выпустить людей и закрыть дверь. Возвращение выполняется в обратном порядке. Новая аварийная заявка не прерывает уже начатый проход.

Ресурсы и допущения

Начально камера пуста, давление нормальное, двери закрыты. Каждое действие двери расходует 1 единицу энергии. Изменение давления расходует 3 единицы энергии, повышение давления — дополнительно 5 единиц воздуха. Перед посадкой проверяется запас на полный проход. При отказе или сигнале выполняется аварийное закрытие; аварийный привод считается исправным и также расходует энергию. Автоматическое спасение оставшихся в камере людей не моделируется.

2 Процессы и разделяемые данные

Процесс	Поведение
Диспетчер	Выбирает ожидающую группу, проверяет ресурсы, выдаёт команды и ждёт завершения действий.
По одному на человека	Ждёт времени подачи заявки, сообщает о готовности, входит и выходит по разрешению.
Внутренняя и внешняя двери	Два процесса выполняют команды открытия и закрытия с задержкой.
Система давления	Отдельный процесс изменяет давление при закрытых дверях.

Для N людей используются N + 4 процесса, включая диспетчер. В примере с тремя людьми — семь процессов. Поведение моделируемых сущностей вынесено в отдельные процессы. Последовательность зависимых действий задаёт диспетчер, но участники могут подавать заявки во время работы оборудования.

Память POSIX

Диспетчер создаёт объект shm_open с O_CREAT | O_EXCL, задаёт размер через ftruncate и отображает его mmap с MAP_SHARED. В Shared находятся шлюз, участники, команды и подтверждения, параметры, PID исполнителей, признаки остановки и время начала. В варианте с неименованными семафорами здесь же находятся объекты sem_t. Указатели на семафоры хранятся локально у каждого процесса и не передаются как общие адреса.

Семафоры

Бинарный семафор LOCK защищает общие состояния и запись целой строки журнала. У каждого исполнителя есть два семафора: REQUEST для команды и FINISHED для подтверждения. ARRIVALS уведомляет о заявке, READY — о подключении, START разрешает общий старт, EXITED подтверждает окончание работы независимого исполнителя.

Диспетчер записывает команду под LOCK, освобождает его, публикует REQUEST и ждёт FINISHED. Исполнитель выполняет действие и подтверждает результат. Ни ожидание команды, ни задержка оборудования не удерживают LOCK. После действий проверяется безопасность и совпадение числа людей в камере с их фактическими состояниями.

Имена и подключение

Объекты называются /akos3_UID_СЕАНС_номер; номер −1 обозначает память. UID разделяет пользователей, имя сеанса — параллельные запуски. Разрешены латинские буквы, цифры и подчёркивания, до 39 символов. Повторное создание существующего сеанса отклоняется. Исполнитель не может занять уже зарегистрированный слот.

3 Три реализации

Уровень требований	Реализация
4–5 баллов	airlock_unnamed: родитель вызывает fork. Неименованные семафоры созданы sem_init с pshared=1 в POSIX-разделяемой памяти.
6 баллов	airlock_named: родитель вызывает fork. Синхронизация через именованные POSIX-семафоры sem_open, данные — в POSIX-памяти.
7–8 баллов	controller и worker запускаются независимо. Именованные POSIX-семафоры и POSIX-память связывают отдельные программы.

Сборка и дочерние процессы

```
make
 ./airlock_unnamed input.txt unnamed.log demo_u
 ./airlock_named input.txt named.log demo_n
```

Makefile создаёт четыре исполняемых файла. MODE выбирается при сборке и определяет точку входа: две программы с fork, независимый диспетчер или исполнитель. Общая логика не дублируется. Флаги -pthread и -lrt подключают необходимые POSIX-средства; пользовательские потоки программа не создаёт.

Независимый запуск

В первом терминале запускается диспетчер:

```
./controller input.txt independent.log demo_i
```

После сообщения Session demo_i ready во втором терминале выполняются команды:

```
./worker demo_i 0 &
 ./worker demo_i 1 &
 ./worker demo_i 2 &
 ./worker demo_i 3 &
 ./worker demo_i 4 &
 ./worker demo_i 5 &
 wait
```

Каждый worker также можно запускать в отдельном терминале. Слоты 0, 1, 2 — внутренняя дверь, внешняя дверь и давление; 3+i — участник с индексом i в файле. Эти процессы запускает оболочка, а не controller. Они открывают существующие память и семафоры по имени сеанса.

Диспетчер ожидает подключения всех N + 3 исполнителей до 60 секунд. После общего старта отсчитываются задержки заявок. При недостающем исполнителе запуск прекращается и ресурсы удаляются. Запуск worker до появления готового сеанса завершается ошибкой подключения.

4 Входные данные и интерфейс

Аргументы программ-диспетчеров: входной файл, новый файл журнала и имя сеанса. Существующий журнал не перезаписывается. Файл input.txt содержит следующий сценарий:

```
2 20 30 200 300 0 42 1 1
3
1 0 0 0
2 1 400 1
3 0 0 0
```

Первая строка: вместимость, воздух, энергия, длительность двери в мс, длительность изменения давления в мс, вероятность отказа в процентах, seed, режим групп и включение аварийного приоритета. Режим групп 1 набирает уже ожидающих людей одного направления до вместимости; 0 обслуживает по одному. Приоритет 1 включает преимущество аварийного возвращения, 0 отключает его.

Вторая строка задаёт число людей. Далее для каждого указаны id, направление, задержка заявки в мс и аварийный признак. Направление 0 — выход в космос, 1 — возвращение. Аварийный признак 1 разрешён только для возвращения. В примере участники 1 и 3 выходят в космос, участник 2 через 400 мс запрашивает аварийное возвращение.

Параметр	Ограничение
Вместимость	1–16
Число людей	1–64
Воздух и энергия	Неотрицательные целые числа
Длительности оборудования	1–60000 мс
Задержка заявки	0–3600000 мс
Вероятность отказа	0–100%
Идентификаторы	Положительные, без повторений

Вход читается функциями libc fopen и fscanf. Вывод через dprintf записывается в стандартный дескриптор вывода и файл журнала. Независимые исполнители открывают тот же журнал с O_APPEND; целостность строк обеспечивает LOCK.

События содержат время [чч:мм:сс], подключения и PID, заявки, состав группы, начало и завершение операций, перемещения, отказ или нехватку ресурсов. В конце выводятся положения и статусы людей, остатки ресурсов, состояние дверей и Safe. Для короткого запуска несколько событий имеют одинаковую секундную метку.

5 Завершение и очистка ресурсов

Обычное завершение

После выполнения всех проходов диспетчер устанавливает общий признак shutdown, пробуждает исполнителей и собирает результаты. В вариантах с fork каждый ребёнок завершается, а родитель вызывает waitpid. Только после этого уничтожаются неименованные семафоры. В независимом варианте исполнители подтверждают завершение общих действий через EXITED и закрывают локальные дескрипторы и отображения.

Сигналы и остановка

SIGINT от Ctrl+C и SIGTERM обрабатываются диспетчером и исполнителями. Обработчик меняет локальный volatile sig_atomic_t; вне обработчика значение переносится в общий атомарный флаг. При инициализации проверяется lock-free реализация общих атомарных флагов. Ожидания используют тайм-ауты и периодически проверяют сигнал. Получение сигнала любым участником останавливает сеанс. Диспетчер завершает текущую команду, прекращает обслуживание, закрывает двери и будит ожидающих исполнителей. При остановке искусственные задержки сокращаются. Положение людей сохраняется; активные заявки становятся interrupted, уже завершённые остаются done. Остановка может оставить людей внутри камеры, но двери должны быть закрыты.

Освобождение объектов

Диспетчер закрывает именованные семафоры sem_close и удаляет их имена sem_unlink. Не именованные семафоры уничтожаются sem_destroy. Все процессы освобождают отображения munmap, а владелец удаляет имя объекта памяти shm_unlink. Частично созданные объекты также очищаются при ошибках входных данных, открытия журнала и запуска.

Исполнители в независимом варианте не являются детьми диспетчера, поэтому waitpid к ним неприменим. После EXITED они больше не изменяют общую модель; удаление имён не отменяет уже открытые ссылки, которые исполнители закрывают перед выходом.

Границы реализации

Поддерживаются обычное завершение, моделируемый отказ, нехватка ресурсов и обрабатываемые SIGINT/SIGTERM. SIGKILL, авария ОС и принудительное уничтожение процесса внутри критической секции не являются штатной остановкой: SIGKILL нельзя перехватить, а бинарный семафор не восстанавливает владельца автоматически. Для нормального завершения используется Ctrl+C или SIGTERM.

Код возврата диспетчера 0 означает полное успешное выполнение, 1 остановку или ошибку. Порядок одновременных заявок и состав групп зависят от планирования процессов. Seed задаёт генератор отказов, но не гарантирует полностью одинаковый журнал.

6 Результаты проверок

Все реализации собраны и проверены в Ubuntu WSL. Сборка выполнена с -Wall -Wextra -Wpedantic -Werror. Дополнительные проверки ошибок запуска выполнены со сборкой -fsanitize=undefined. Предупреждений компилятора и сообщений санитайзера в этих проверках не получено.

Проверка	Результат
Обычный запуск всех трёх реализаций	Выполнено 3/3, камера пустая, двери закрыты, Safe: 1; код 0.
Режим по одному и приоритет выключен	Все три реализации завершают 3/3, Safe: 1; код 0.
Отказ 100%	Выполнено 0/3, воздух 20, энергия 30, двери закрыты; код 1.
Начальная энергия 0	Остановка до посадки, камера пустая, энергия 0; код 1.
SIGINT и SIGTERM диспетчеру	Во всех реализациях остановка с закрытыми дверями, Safe: 1; код 1.
SIGINT независимому исполнителю	Остановлен весь сеанс, двери закрыты, Safe: 1; код 1.

В проверенных обычных запусках двух вариантов с fork получены воздух 15 и энергия 9; в независимом — воздух 15 и энергия 16. Разница объясняется формированием групп: совместный проход расходует меньше энергии. Это допустимое следствие асинхронного поступления заявок.

При проверках сигналов примерно через 250 мс в камере оставались два человека; воздух 20, энергия 28. Завершение не отмечало их как прошедших. После всех 22 основных запусков проверено отсутствие объектов соответствующего сеанса в /dev/shm.

Дополнительно проверены неверная вместимость, защита существующего журнала, SIGINT группе процессов, повторный слот исполнителя, конфликт имён сеанса, остановка неполного независимого запуска и истечение 60-секундного ожидания исполнителей. После этих сценариев также проверена очистка IPC-объектов.

Использование ИИ

Текст отчёта был сформирован ИИ (ChatGPT) на основе требований задания и исходного кода программы, включая описание структуры решения, взаимодействия процессов. ChatGPT также подготовил оформление документа.